

Lecture 4

Programming Paradigms

Example Code and Exercises

<https://github.com/eecs498-software-design/lecture>



Paradigms

Imperative

- Specify what you want the program **to do**.
- Example: C++ find even numbers

```
int main() {  
    vector<int> numbers = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10};  
    vector<int> evenNumbers;  
  
    for (int i = 0; i < numbers.size(); ++i) {  
        if (numbers[i] % 2 == 0) {  
            evenNumbers.push_back(numbers[i]);  
        }  
    }  
}
```

Declarative

- Specify what **result** you want.
- Example: SQL find even numbers

```
SELECT value  
FROM numbers_table  
WHERE value % 2 = 0;
```

Paradigms

Imperative

- Specify what you want the program **to do**.

Declarative

- Specify what **result** you want.

CAUTION: Not the same distinction as “what vs. how” à la abstraction.

- Abstraction is about the level of detail.
- Imperative vs. Declarative is about the way you specify it.



Structured Imperative

Non-Structured

- Control flow can go anywhere.

```
void sum_evens_non_structured(const vector<int> &data) {  
    int sum = 0;  
    int i = 0;  
  
loop_start:  
    if (i >= size) goto loop_end;  
  
    if (numbers[i] % 2 != 0) goto next_iteration;  
  
    sum += numbers[i];  
  
next_iteration:  
    i++;  
    goto loop_start;  
  
loop_end:  
    cout << "Total: " << sum << endl;  
}
```

Structured

- Constrained control-flow structures.

```
void sum_evens_structured(const vector<int> &data) {  
    int sum = 0;  
    for (int i = 0; i < data.size(); i++) {  
        if (data[i] % 2 == 0) {  
            sum += data[i];  
        }  
    }  
  
    cout << "Total: " << sum << endl;  
}
```

Procedural Imperative

Non-Procedural (Structured)

```
void sum_evens_structured(const vector<int> &data) {  
    int sum = 0;  
    for (int i = 0; i < data.size(); i++) {  
        if (data[i] % 2 == 0) {  
            sum += data[i];  
        }  
    }  
  
    cout << "Total: " << sum << endl;  
}
```

Procedural

- Split into self-contained functions.
- Each has own scope, resources, etc.

```
function is_even(int x) {  
    return x % 2 == 0  
}  
  
void sum_evens_structured(const vector<int> &data) {  
    int sum = 0;  
    for (int i = 0; i < data.size(); i++) {  
        if (is_even(data[i])) {  
            sum += data[i];  
        }  
    }  
  
    cout << "Total: " << sum << endl;  
}
```

Structured Imperative

Non-Structured

- Control flow can go anywhere.

```
void sum_evens_structured(const vector<int> &data) {  
    int sum = 0;  
    for (int i = 0; i < data.size(); i++) {  
        if (data[i] % 2 == 0) {  
            sum += data[i];  
        }  
    }  
  
    cout << "Total: " << sum << endl;  
}
```

Structured

- Constrained **control-flow structures**.

```
void sum_evens_non_structured(const vector<int> &data) {  
    int sum = 0;  
    int i = 0;  
  
loop_start:  
    if (i >= size) goto loop_end;  
  
    if (numbers[i] % 2 != 0) goto next_iteration;  
  
    sum += numbers[i];  
  
next_iteration:  
    i++;  
    goto loop_start;  
  
loop_end:  
    cout << "Total: " << sum << endl;  
}
```

Paradigms: Object Oriented

- Generally builds onto procedural, structured imperative.
- Core ideas:
 - **Encapsulation** of data and operations together
 - Interact with *objects* via “message passing” (i.e. `obj.func()`)
 - *Objects* may be mutable
 - *Objects* may or may not map to real-world entities
 - Generally uses subtype polymorphism.



Paradigms: Functional

- Generally builds onto a declarative approach to specify desired results as expressions/functions.
- Core ideas:
 - Functions as operations performed on data (not encapsulated)
 - Avoid state changes and side-effects
 - Prefer immutable data – compute new data, don't change in-place.
 - Generally uses ad hoc polymorphism.



Demo: Processing Arrays



Exercise: Processing Arrays

- Search for “mdn javascript arrays” – reference page
- Write the following as iterative (loop) and functional (no loop):
 - Find number of even elements in an array.
 - Get a list of only the even elements in an array.
 - Square each element in an array.
 - Find the sum of elements in the array.



Demo: Electronic Exam Questions

Object-Oriented vs. Functional

